

Practical 5: 0/1 Knapsack Problem

Implementation using Dynamic Programming in C++

Source Code

```
#include <iostream>
#include <vector>
using namespace std;

int knapsack(int W, vector<int>& weight, vector<int>& value, int n) {
    // dp[i][w] = maximum value using first i items with capacity w
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weight[i - 1] <= w) {
                // Option 1: exclude item i-1
                // Option 2: include item i-1
                dp[i][w] = max(dp[i - 1][w],
                               dp[i - 1][w - weight[i - 1]] + value[i - 1]);
            } else {
                // Item can't fit, so exclude it
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    return dp[n][W];
}

int main() {
    int n, W;

    cout << "Enter number of items: ";
    cin >> n;

    vector<int> weight(n), value(n);

    cout << "Enter weights of items:\n";
    for (int i = 0; i < n; i++)
        cin >> weight[i];

    cout << "Enter values of items:\n";
    for (int i = 0; i < n; i++)
        cin >> value[i];

    cout << "Enter knapsack capacity: ";
    cin >> W;

    int maxValue = knapsack(W, weight, value, n);

    cout << "Maximum value that can be obtained: " << maxValue << endl;

    return 0;
}
```

How It Works

The array `dp[i][w]` stores the maximum value achievable using the first `i` items with a knapsack capacity of `w`. For every item, the algorithm either **includes** it (if it fits within the remaining capacity)

and adds its value to the best solution for the remaining capacity, or **excludes** it and carries forward the best value without it. The final answer is stored at `dp[n][W]`.

Sample Run

```
Enter number of items: 4
Enter weights of items:
1 3 4 5
Enter values of items:
1 4 5 7
Enter knapsack capacity: 7
Maximum value that can be obtained: 9
```

Complexity

Both time and space complexity are $O(n \times W)$, where n is the number of items and W is the knapsack capacity.